ChatGPT

# Deterministic Interactive Computation in the Browser

## Abstract

Interactive software is notoriously difficult to reproduce due to unpredictable user inputs, timing, and environment effects. We present **RedByte**, a browser-based digital logic simulation system that achieves **deterministic interactive computation** by treating determinism as a first-class semantic contract. RedByte guarantees that given the same initial state and the same sequence of user events, the simulation will always reach the same final state [1]. This guarantee is enforced through *event-bounded execution* and *canonical state representation*, with cryptographic hashing (SHA-256) to **verifiably prove** that a replayed session is identical to the original [2]. The system supports *time-travel navigation*: users can step forward and backward through a recorded session, inspecting state at any event boundary with stable, repeatable results [3]. A built-in developer panel allows recording sessions, replay verification, and state inspection, all within the browser. We validate RedByte's determinism contract via comprehensive automated tests and live operational verification in multiple browsers. Our results demonstrate that browser-based interactive applications can be made **fully deterministic and replayable**, enabling new possibilities in debugging, education, and collaborative sharing of interactive computations.

## Introduction

Modern interactive applications (e.g. visual simulations, GUI-based tools) often suffer from **nondeterministic behavior** – outcomes may vary across runs due to timing of events, user input ordering, or hidden state. This nondeterminism complicates debugging and collaboration: a subtle bug might only appear under specific interaction timing, making it hard to reproduce, or a user's demonstration cannot be exactly replayed by others. Traditional debugging techniques address this with heavy-weight record/replay at the system level (e.g. Mozilla's *rr* tool which records all nondeterministic inputs to a process [4], or Microsoft's Time-Travel Debugging which traces program execution for reverse stepping [5]). Meanwhile, application-level approaches like Elm and Redux encourage structuring code so that replaying the same input events yields the same state [6] [7]. However, few interactive systems treat determinism as an **explicit contract** baked into their design.

**RedByte** is a browser-based digital logic simulator that makes determinism a cornerstone of its architecture. RedByte's core claim is simple: *"Given the same initial circuit state and the same ordered sequence of recorded events, the simulation produces the same resulting state."* [8] In other words, interactive sessions are **100% reproducible** – any divergence is considered a bug. To our knowledge, RedByte is the first browser-resident interactive system to provide **cryptographically verifiable replay** of user sessions: the final state of a "live" execution and a replayed execution can be compared via their SHA-256 hashes, which will match if and only if the runs were identical [2]. Crucially, RedByte achieves this determinism *without* resorting to artificial timing controls or single-threaded hacks; instead, it defines a **semantic determinism contract** at the application level and implements mechanisms to uphold it.

The contributions of this work include: (1) a formal **Determinism Contract** defining guarantees and non-goals for interactive computation, (2) a set of implementation techniques – event logging, state normalization, cryptographic hashing, and time-travel inspection – that realize this contract in a browser environment, and (3) an evaluation demonstrating that a complex in-browser simulation can be made deterministic and replayable in practice. RedByte's approach brings together ideas from programming languages (referential transparency, state snapshots), systems (record/replay logging), and human-computer interaction (time-travel UI for debugging) into a unified solution. This work is *novel* in treating determinism not as a low-level debugging trick or an afterthought, but as a **first-class, user-visible property of an interactive system**. By locking down the execution semantics and exposing replay and rewind capabilities to developers, RedByte enables new workflows: for example, a bug in a circuit can be demonstrated by sharing a recorded session log, or a student's interactive simulation can be "time-traveled" step by step by an instructor to examine its behavior.

In the rest of this paper, we detail the design and implementation of RedByte and how it achieves deterministic interactive computation. We first discuss background and motivations for pursuing strict determinism in a single-user browser app. We then present the RedByte **Determinism Contract**, which precisely scopes what is (and isn't) guaranteed. Next, we describe the implementation over four milestones (A–D), covering the progression from core deterministic primitives to a full-fledged record/replay and time-travel system. We validate RedByte's guarantees with both automated tests and live demonstrations. Finally, we compare our approach to prior work in debugging, state replay, and educational simulations, and discuss the limitations and future directions of RedByte's determinism-first design.

## Background and Motivation

**Why determinism?** Deterministic execution has long been a goal in contexts like concurrent programming and distributed systems, but it is equally valuable in interactive single-user applications. In an interactive setting, *determinism* means that a user's session can be captured and later replayed to yield exactly the same states and outputs. This has clear benefits for debugging (the "Heisenbug" that only occurs on Tuesdays can be recorded and studied), for testing (regressions can be identified by replaying canonical event sequences), and for sharing interactive experiences (a designer's complex sequence of actions can be sent to a colleague as a replayable artifact, not just a video).

Despite these benefits, few mainstream interactive tools emphasize full determinism. Traditional GUIs and simulators often rely on imperative state and real-time event handling, which can introduce hidden nondeterminism. Even subtle factors like iteration order in an unsorted data structure or reliance on the wall-clock time can cause two runs to diverge. In educational circuit simulators (e.g. Logisim or CircuitVerse), one can save a circuit's static design, but there is usually no built-in facility to record and replay the *interactive session* of toggling inputs and observing outputs; the moment-by-moment behavior is ephemeral. If a student discovers an odd glitch after a series of steps, they often cannot precisely retrace those steps later.

Time-travel debugging interfaces have demonstrated the value of capturing application state over time. The Elm language's Time-Travelling Debugger and Elm Reactor, for instance, record all inputs (user events) to a pure-functional Elm program and allow a developer to move backward and forward in the event history [6] [9]. Redux DevTools offers similar functionality for Redux-based React applications, logging every dispatched action and allowing developers to "rewind" the Redux store to a previous state [7]. These tools rely on the **application logic being pure** or otherwise well-behaved: given the same sequence of actions,

the state should be the same. In Elm this is ensured by design (no mutable state outside the Elm runtime), whereas in Redux it's a strong convention (reducers should be pure functions). RedByte's work builds on this idea but goes further – *mandating* purity/determinism at the system level and actively verifying it. Instead of assuming developers won't introduce side effects, RedByte defines a strict log of all state-changing events and uses hashing to confirm that re-execution of that log yields no surprises.

Another line of relevant work is **record/replay debugging** in low-level systems. Tools like *rr* for Linux record a program's execution (capturing syscalls, signals, thread scheduling, etc.) so that it can be deterministically replayed under a debugger [4]. Microsoft's Time-Travel Debugging (TTD) for Windows similarly allows one to step backwards in a debugging session by recording execution traces [5]. While powerful, these systems operate at the machine instruction level and primarily target reproducibility for debugging crashes or race conditions. They are not intended to be user-facing features of applications, and they come with significant overhead (logging huge traces to capture all nondeterministic inputs). By contrast, RedByte operates at the **application semantic level**: it knows the domain (digital circuits simulation) and logs only high-level events relevant to that domain (e.g. a user toggling an input switch, or the simulator advancing one tick). This focus yields a minimal, self-contained event log that is *sufficient* to reproduce the session without recording every low-level detail. It's a higher-level, more lightweight approach appropriate for a browser environment and even end-user use in the future.

In summary, prior work shows that deterministic replay is *desirable* and *feasible* under certain conditions – whether by restricting programming models (Elm), using architectural patterns (Redux), or heavy instrumentation (rr, TTD). **RedByte's motivation** is to bring deterministic replay to a setting that is both **richly interactive** and **widely accessible** (runs in a standard web browser), without sacrificing rigor. We treat determinism as a contract with the user/developer: if you stay within the specified semantics, the system will never surprise you with a different outcome on replay. This contract is described next, and it guides both what RedByte does and what it intentionally does *not* do.

## Design: The RedByte Determinism Contract

The design of RedByte is governed by a formal **Determinism Contract** that specifies the exact guarantees of the system, as well as its scope and limitations. In essence, the contract defines *what it means* for RedByte to be deterministic and how that is achieved in a browser-based logic simulator context. We summarize the key points of the contract here.

**Scope of Determinism:** RedByte's determinism applies to the *logic simulation engine* and any *recorded execution sessions* (event logs). If a user builds a circuit and interacts with it (toggling inputs, stepping the simulation, etc.), those interactions can be recorded; RedByte guarantees that replaying that recorded session on the same initial circuit through the same engine will produce identical results. Importantly, certain aspects of the system are deliberately *not* covered by the determinism contract: the UI rendering and animation are not deterministic (nor need they be), performance timing is not considered, and any external I/O or integration (network calls, etc.) is outside scope [10] [11]. Determinism is focused on the *semantic state* of the simulation (the logical state of circuit nodes, signals, etc.), not pixel-perfect rendering or real-time behavior. The contract's "Unit of Determinism" is precisely defined as:

> **Given the same initial circuit state and the same ordered sequence of recorded events, the simulation produces the same resulting state.** [8]

Everything else in the design stems from this fundamental guarantee. To make it actionable, the contract spells out specific properties that RedByte upholds:

- **Deterministic Execution:** If two runs start from an identical circuit configuration, process the same sequence of events (and are running under the same engine version), they will end in the same state [1]. The contract even expresses this formally: for all circuits and event sequences, `hash(execute(circuit, events))` remains invariant [12]. In practice, this means no matter how many times or on which machine you replay a given RedByte session (as long as the engine version is compatible), the outcome is byte-for-byte identical.

- **Verifiable Replay:** RedByte makes the above claim testable by hashing the simulation state. The final state of a live execution is hashed (using SHA-256), and the final state of a replayed execution is hashed; if the hashes are equal, the execution is deemed identical [2]. Because SHA-256 is collision-resistant and our state serialization is canonical, `hash(live) == hash(replay)` serves as a *cryptographic proof* of deterministic behavior [13]. This verifiability is a cornerstone of the contract – it's not just that "we believe it's deterministic," but that anyone can independently confirm determinism by comparing hashes. The system even provides tools to do this comparison live during development (see Section *Implementation*).

- **Stable Time Travel:** Beyond just final-state determinism, RedByte guarantees that *every intermediate state* at each event boundary is reproducible and can be navigated consistently [3]. If a session had, say, 50 events (including the initial circuit load as an event), then the state after event 10 or after event 37 can be revisited. Stepping the simulation forward one event at a time (from t to t+1) or backward (t to t-1) yields the same sequence of states every time [14]. In other words, not only is the end result the same, but the *path* through state space is the same on every replay. The contract emphasizes that time-travel is implemented as a pure **query** over the recorded log – it does not alter the log or create divergent histories [15]. This means one cannot "change the past" during replay; one can only inspect it. This design keeps the determinism guarantees clean, since allowing state edits during time travel would introduce new event sequences.

- **Exportable Reproducibility:** To ensure the determinism is useful beyond a single browser session, RedByte allows any recorded session to be exported as a JSON artifact [16]. An exported log contains the initial circuit description, the event log (with a version number), and some metadata like timestamps and event count [17]. The determinism contract guarantees that this exported bundle is sufficient to reproduce the execution on any other compatible instance of RedByte [18]. In other words, the JSON log is a **portable witness** of the interactive computation. This is akin to a research experiment's raw data: anyone with the log can rerun the "experiment" and should observe the same outcomes. By including a version and metadata, the system prepares for future changes (so older logs can be recognized and handled appropriately) and aids debugging (e.g., knowing how many events were in the session).

Equally important is what the **Determinism Contract explicitly does *not* guarantee**. RedByte does *not* promise real-time performance or consistent timing – two identical replays might take different wall-clock time or frame counts, but that doesn't violate determinism since the logical results match [19]. It does not guarantee cross-version determinism unless the version is the same; if we update the simulation engine in an incompatible way, an old event log might produce a different result, and that's considered outside the contract (hence the versioning of logs to detect this) [20]. The contract is careful to note that **cryptographic**

**integrity** is not provided [21] – while we use hashes to verify identity of execution, we do not sign logs or prevent tampering. The logs are assumed to be trustworthy inputs; a maliciously edited log could yield a valid (but different) execution without detection, as long as the hash comparison is only used to compare a replay to its own original execution. Similarly, determinism across different implementations of the engine is not guaranteed [22] – only the reference engine (the one in RedByte's codebase) is assured to follow the contract. UI behavior is not deterministic (the contract explicitly excludes the user interface and rendering aspects) [23], and any potential multi-user/networked operation would introduce complexities that are out of scope [24]. These exclusions keep the contract **focused** and realistic. They align with RedByte's positioning: it's not a distributed simulation or a security system, it's a single-user local simulation environment with a strong reproducibility guarantee.

**Design Choices to Enforce Determinism:** To uphold the contract, RedByte's design incorporates several strategies to eliminate sources of nondeterminism (somewhat analogous to eliminating data races in a parallel program). The contract document enumerates common risks and how they are mitigated [25]. For instance, JavaScript's iteration order for object properties or `Map/Set` is not strictly guaranteed, which could lead to inconsistent ordering of circuit elements in memory. RedByte addresses this by implementing a **canonical state normalization**: every collection of nodes, connections, or other elements is sorted in a stable order before computing a hash or performing a comparison [26] [27]. This ensures that two logically identical circuits produce the exact same serialized form. Another risk is reliance on real-time clocks – if the simulation calls `Date.now()` to timestamp events, two runs could differ by a few milliseconds. RedByte solves this by **injecting timestamps** as part of the event data: the event log carries its own time information, and the engine uses those timestamps instead of calling the clock during replay [28]. In effect, time becomes just another input in the log, under our control, rather than an uncontrolled external variable.

RedByte also avoids hidden sources of nondeterminism: the determinism-critical code has no random number generation (no use of `Math.random` or non-deterministic APIs except for the cryptographic hash which is used in a controlled way) [29]. The simulation logic deals only with integer values (logic 0/1) and avoids floating-point arithmetic to prevent issues with precision or platform-specific math differences [29]. All state changes in the simulation are made explicit as logged events, so there is no "implicit state" that could diverge without being noticed [30]. Data structures are copied or frozen to avoid unintended mutation through shared references [31]. In summary, RedByte's design borrows from best practices in functional programming and state snapshotting to remove the usual suspects of nondeterminism. This disciplined design is what allows the bold determinism claims to hold in practice.

Finally, the **Determinism Contract is treated as a binding specification** for the system. It is versioned (currently v1.0) and any future changes that could affect determinism semantics must result in a new version of the contract and the event log format [32] [33]. The RedByte team treats this contract like a contract with users: once declared "locked," no breaking changes will be introduced without explicit opt-in to a new version. This is akin to how programming languages define a memory model or how network protocols are versioned for compatibility. By doing so, RedByte ensures that deterministic behavior is not accidentally compromised by future feature additions – a lesson learned from many systems where a small change can re-introduce nondeterministic behavior if not checked.

In summary, the RedByte Determinism Contract sets the stage by defining *what determinism means* for our interactive browser-based system, the **guarantees** (same inputs = same outputs, verifiable by hash, navigable history, reproducible exports) and the **non-goals** (performance, cross-version, external effects,

etc.). With this contract in place, we next describe how the system was implemented to satisfy it, organized through a series of developmental milestones (A through D) that incrementally built up RedByte's deterministic capabilities.

# Implementation

Implementing deterministic interactive computation in RedByte involved four major milestones, each corresponding to a set of features that uphold different aspects of the determinism contract. We describe each milestone and the resulting system components, highlighting how they map to the guarantees from the contract.

## Milestone A: Core Deterministic Primitives

**Milestone A** established the **foundational primitives** for determinism: a deterministic state representation, a hashing mechanism, and an event log format. Before this milestone, RedByte had no way to verify or persist deterministic behavior – it was just a goal. Milestone A delivered the *mathematical and structural basis* on which all later features build [34].

The first key primitive is **state normalization**. This is implemented as a function that takes a circuit's state (the set of logic nodes, their connections and any dynamic state like signal values) and produces a *canonical, sorted representation* of that state [35] [36]. All nodes are sorted by a stable identifier, all connections are sorted, and nested structures (like configuration settings on a node) are turned into sorted key-value forms. By normalizing the state, RedByte ensures that if two circuits are structurally identical, their normalized representations are exactly identical byte-for-byte. This addresses issues like JavaScript's iteration order variability: for example, without normalization, two identical sets of connections might be iterated in different orders and produce different hash outputs or JSON strings. With normalization, the order is fixed deterministically [37] [26]. The importance of this step is hard to overstate – it removes an entire class of potential nondeterminism stemming from the language/runtime. In the determinism contract, this directly mitigates the "non-deterministic iteration" risk [27] and contributes to guaranteeing deterministic execution (Contract §3.1) by making state comparisons meaningful and stable [38].

On top of normalized state, RedByte implements **deterministic hashing**. Specifically, it uses the Web Crypto API in the browser to compute a SHA-256 hash of the normalized circuit state [39] [40]. This function returns a promise (since Web Crypto is asynchronous) that resolves to a hex-encoded hash string. The use of SHA-256 ensures the hash is collision-resistant and thus can serve as a reliable fingerprint of the state [41]. Some design choices here were: using a browser-native API (for compatibility; Node's `crypto` module would not work in a browser without polyfills) [40], and choosing an output format (hex) that is human-readable and portable. With `hashCircuitState()` implemented, any two states can be compared quickly; in particular we can compare the hash of a final state from one run to that of another run to test for equality. This hashing capability underpins the **Verifiable Replay** guarantee (Contract §3.2) [42]. It was important to verify that the hashing itself is deterministic (the same state always produces the same hash, and no variation across browsers) – Milestone D later covers the validation of Web Crypto for this purpose [43] [44].

The third piece of Milestone A is the **Event Log format**. RedByte defines a structured, versioned format for recording events that occur during a simulation session [45]. In the initial version (EventLogV1), the log contains an array of `Event` objects and a version number. Each event is a typed record; for example, one

event type is `circuit_loaded` (capturing the initial load of a circuit), another is `input_toggled` (when a user flips a switch or toggles an input node), another represents a simulation tick (progression of the simulation engine by one timestep), and another might represent a node's internal state change [46]. Every event includes a timestamp and relevant data (e.g. which node was toggled, or the state that was changed). This event log is **append-only** – events are recorded in order and never retroactively modified [47]. It is also **self-describing**: because each event has a `type` field and all necessary info, the log can be interpreted on its own, without additional hidden context [47]. Crucially, the log carries a version number (`version: 1` for now), which future-proofs the format: if we need to change how events are recorded, we would introduce EventLogV2 and maintain backward compatibility for V1. The presence of a stable event log format directly supports the **Exportable Reproducibility** guarantee (Contract §3.4) – by having a complete and versioned history of what happened, we can export it and later import it to replay the session [48].

Finally, Milestone A introduced formal **type definitions** in the code (using TypeScript) for all these determinism-related structures [49]. This includes types for the normalized state, the event log and event entries, and the results of verification. These types act as a machine-checkable version of the contract – they ensure at compile time that functions only accept data in the expected form, and they help developers use the determinism API correctly. While this is more of an implementation detail, it contributes to reliability (it's harder to accidentally treat non-normalized data as normalized, for example) and serves as living documentation for the contract.

Milestone A did **not** include any actual recording or replay functionality – it was purely the foundation. According to the plan, features like actually capturing events or replaying them were slated for Milestone B [50]. Likewise, user interface integration or time-travel controls were future work at that point [50]. This separation ensured that first the team got the *ground truth* right (state, hash, log format) before building higher-level features. Milestone A's deliverables map to specific sections of the determinism contract: it enabled deterministic execution by making state comparisons invariant [38], it enabled verifiable replay by providing a hash [51], and it enabled reproducible logs by defining the event log format [52]. Tests were written to verify each primitive: e.g., tests to ensure two identical circuits give the same normalized output, that hashing the same circuit twice yields the same hash, that the event log can serialize to JSON properly, etc [53] [54]. By the end of Milestone A, RedByte had the **mathematical foundation** in place: any divergences in state could be detected, and any session could in theory be recorded and later replayed (the data structures to hold a session were defined, even if the act of replaying was not yet implemented).

## Milestone B: Record & Replay Mechanism

If Milestone A established *what* to compare for determinism, **Milestone B** made the system actually *do something* with that foundation – namely, **record** interactive sessions and **replay** them, and then verify that the replay matched the original. In short, Milestone B operationalized determinism.

One major component introduced was the **Recorder** module [55] [56]. The RedByte recorder hooks into the simulation environment to capture every relevant event as the user interacts. This includes when a circuit is initially loaded (the recorder logs a `circuit_loaded` event with the initial circuit state), whenever the user toggles an input or a UI control that affects the circuit (`input_toggled` events with which node/port changed and the new value), whenever the simulation advances in time (`simulation_tick` events, which might occur on a clock cycle or when a "step" button is pressed), and any explicit changes to node state (for example, if there's a widget that changes a node's configuration) [46]. The recorder provides functions like `recordInputToggled(nodeId, portName, value)` for the codebase to call whenever such an action

occurs [57]. It appends a new event object to the in-memory log each time. Because the event format was defined in Milestone A, implementing the recorder was straightforward in terms of data schema – the key was to ensure it was used in all the right places in the app and that it captured the needed information.

Design-wise, the Recorder is built to have **minimal performance impact and no side-effects** on the simulation itself [58]. It essentially listens and logs, but does not alter how the simulation behaves. The events are timestamped (using an injected clock source, usually just `Date.now()` at the time of recording) [59]. Those timestamps are part of the log for informational purposes and for potential use by the engine (for example, a `simulation_tick` event might carry a `dt` or time delta that the engine uses). Because the recorder is append-only and writes to an in-memory list, it's quite simple: it can start, stop, and provide the accumulated `EventLog` when needed [60]. Tests for the recorder ensured that it correctly captures sequences of events and that after stopping, the log remains immutable (so you capture exactly what happened, then freeze it) [61]. The recorder essentially fulfills the role of "making a trace" of the execution, similar to rr's trace but at a much higher level of abstraction (user events, not CPU instructions).

The counterpart to recording is **Replay**. Milestone B added a `runReplay()` function that takes an `EventLog` and an initial circuit, and executes the log step by step [62] [63]. Since RedByte is a simulation of logic circuits, executing an event generally means updating the circuit's state or advancing the simulation engine. The replay engine uses the exact same logic engine that drives live simulations, but feeds it events deterministically. For example, when replay encounters an `input_toggled` event, it finds the corresponding node in the circuit and sets its input to the specified value (emulating what the user originally did); when it encounters a `simulation_tick` event, it calls the engine's tick function with the same parameters (delta time, tick count) as originally recorded [64]. Because the original run and the replay run use the same sequence of operations on the same initial state, the contract predicts they end in the same final state [65]. Replay was implemented to be **pure** and stateless: given the log and the initial circuit, it doesn't depend on anything else (like no external input or global state) and always produces the same result [65]. This purity is essentially the embodiment of Contract §3.1 (Deterministic Execution) in code: it ensures same events → same outcome.

To make verifying determinism easy, Milestone B also introduced a **Verification Command** called `verifyReplay()` [66] [67]. This is a convenience function that automates the process of checking a recorded session. Internally, `verifyReplay` does two executions: it takes the initial circuit and event log, and first *replays* it using `runReplay()` to get a final circuit state. But it also independently "re-applies" the events in a fresh instance of the circuit in the same order (essentially mimicking what happened live) to get a second final state [68]. It then hashes both final states (using the Milestone A hashing) and returns a result indicating whether the hashes match [68]. The idea of doing two executions (one via the replay function, one via a direct application) is to double-check that our replay logic truly mirrors a real execution. If our recorder or replay had a bug (say it missed an event or replayed something incorrectly), the hashes would diverge and `verifyReplay` would catch it by reporting `equal: false`. In the normal (correct) case, `verifyReplay` yields `equal: true` with identical `liveHash` and `replayHash` values [69] [70]. The output of `verifyReplay` thus provides a concrete confirmation of determinism for that session. This command corresponds to the **Verifiable Replay** guarantee of the contract [71] and was a critical piece for both testing and later for the UI.

Additionally, Milestone B implemented **session export** functionality so that the recorded `EventLog` (plus initial circuit) can be saved out of the application [72]. This was done via a simple function or UI control to serialize the log to JSON (using the structure defined in Contract §3.4) and trigger a download in the

browser [73]. The exported JSON looks like the format we described earlier: it includes the circuit, the events array, and some metadata (like how many events, and an export timestamp) [73]. Exporting is useful for sharing or archiving a session, and it's the realization of the "exportable reproducibility" promise in a user-visible way. One can take the JSON file, later load it (Milestone D included the ability to import, tested manually [74] [75]), and then run `verifyReplay` or replay it in the UI to see the same behavior.

After Milestone B, RedByte had a fully deterministic *backend*: you could record any sequence of interactions and programmatically verify and export that sequence. At this stage, these features were still aimed at developers (there wasn't a nice GUI for a layperson to do it, until Milestone C), but the core system was in place. The **guarantees addressed by Milestone B** map clearly to the contract: it implemented deterministic execution (by ensuring replay is faithful to live runs) [76], verifiable replay (via hash compare in `verifyReplay`) [77], and exportable logs (via the export JSON) [78] [71]. It also continued mitigating risks: for example, to handle the "wall-clock time" issue, the recorder in B made sure to log timestamps so that replay doesn't call `Date.now()` at all – it uses the recorded timestamp [79]. Also, by logging *every* state mutation as an event, we ensure there's no hidden state change that isn't captured [79]. Tests in Milestone B covered scenarios like "if two different event logs produce different final states" (ensuring no false positives of equality), that `verifyReplay` indeed catches divergences, and that the recorder can be stopped/started without issues [61] [80]. By the end of this milestone, the system's determinism was no longer just theoretical; it was demonstrable through cryptographic evidence.

## Milestone C: Time-Travel Inspection and Debugging UI

With reliable record & replay in place, **Milestone C** turned to making determinism *inspectable* and developer-friendly. The focus was on **Time Travel** capabilities: the ability to navigate through a recorded event log, observe state at various points, and visualize differences between states. Milestone C also introduced a developer-facing UI panel to drive these features in the browser.

A core addition was the **State Inspector** API, specifically a function `getStateAtIndex(initialCircuit, eventLog, index)` which returns a snapshot of the circuit state after replaying the first N events (up to the given index) [81]. This essentially wraps the replay function to stop at an intermediate point. If `index = 0`, it would give the initial state; if `index = eventLog.length` (the end of the log), it gives the final state (same as a full replay). Any intermediate index gives the state after that event has been applied. This capability realizes the "inspect any event boundary" part of the contract [3]. The snapshot returned includes not only the circuit state but also metadata like which event index it corresponds to and the timestamp of that event [82]. This proved helpful to display context (e.g., "State at event 5 of 20, timestamp T"). The `getStateAtIndex` function is deterministic and idempotent – calling it multiple times with the same inputs yields the same snapshot [83]. Under the hood, it literally performs a replay up to that index and then clones the state for output. Though replaying from scratch for each query could be slow for large logs, the design prioritized correctness over optimization (and logs are typically not huge for logic simulations). The contract's stable time travel guarantee is satisfied because this function will always return the same state for the same index [83].

Building on the inspector, **navigation primitives** were added: `stepForward()` and `stepBackward()` which take the current snapshot and move one event forward or back [84] [85]. These are convenience functions so that a user (or the UI) can easily iterate through the event history without manually specifying indices. Internally, `stepForward` just calls `getStateAtIndex` with index+1 relative to the current snapshot's index (and similarly for backward). They return a new snapshot or `null` if you're at the end or

beginning of the log [86]. Importantly, these were designed such that forward then backward (or vice versa) brings you back to the original snapshot [87]. This might sound obvious, but it required careful handling to ensure no hidden side-effects: e.g., the simulation engine is reset properly when stepping backward so that you truly re-derived the previous state rather than trying to "undo" operations (RedByte does not do inverse operations; it always reconstructs by replay). Tests confirm that `stepForward` followed by `stepBackward` yields the identical state you started with [88]. These navigation functions, combined with `canStepForward()` predicates, give a simple interface for a UI to allow time travel.

Another feature in Milestone C is **state diffing**. To help users understand what changed between two points in time, RedByte includes a `diffState(beforeSnapshot, afterSnapshot)` utility [89]. This function compares two circuit state snapshots and produces a structured diff: which node states changed (and how), which signal values changed, if any nodes were added or removed, or connections changed [90] [91]. In the context of a logic circuit, a "node state change" could be something like a flip-flop changing its stored bit from 0 to 1, a "signal change" might be a wire going high or low. By providing a diff, RedByte helps pinpoint the effects of each event. For example, if stepping from event 4 to event 5 toggled an input, the diff for that step will show that that input's signal changed (and perhaps downstream signals changed due to propagation). This is very useful for debugging and education – it provides a concise explanation of "what just happened" when you step through the log [92]. While the diff capability is not an explicit part of the contract guarantees (the contract doesn't *require* a diff, it just requires time-travel inspection), it is a supporting feature to make the time-travel usable. It aligns with the spirit of making the interactive computation intelligible and transparent.

Finally, Milestone C delivered the **Determinism Dev Panel**, a UI component accessible in the browser (triggered by a hotkey, e.g. Ctrl+Shift+D) [93] [94]. This panel is essentially a GUI front-end for all the features from Milestones A, B, C. It provides buttons and indicators for: - **Recording**: a button to start/stop recording a session, with an indicator when recording is active [95]. - **Verification**: a button to run `verifyReplay` on the recorded log and display the result (for instance, showing the live hash and replay hash and a "✓ Deterministic" if they match) [96]. This gives immediate feedback to the developer that the run was deterministic. If something were wrong (hash mismatch), it would show a failure. - **Export**: a button to export the log JSON for download [96]. - **Time Travel controls**: once a log is verified (to ensure we only time-travel on known-good logs), an "Initialize Time Travel" button becomes available, which sets up the internal state for navigation [97]. Then the panel shows "Step Back" and "Step Forward" buttons to move through events, along with a counter like "Event X of Y" to indicate the current position [97]. These buttons call the navigation functions under the hood. They are disabled appropriately at the start or end of the log (so you can't step past bounds). There's also a Reset button to exit time-travel mode and clear the session.

The determinism panel is designed as a **developer tool** (similar to Redux DevTools, not meant for end-users in production) [98]. It favors function over form – minimal styling, just enough UI to expose the functionality. It logs internal state changes to the console for transparency [98]. The idea is that a RedByte developer or power user can open this panel at any time while using the Logic Playground (the main app interface) and see determinism in action: record something, verify it, step through it. This serves as both a debugging aid and a **live demonstration of the determinism contract**. In fact, the contract document explicitly cites the dev panel as part of the proof of correctness (Contract §6) – it's a way to let anyone verify the claims interactively [99] [100].

At the end of Milestone C, RedByte's determinism was not only implemented but also **accessible and observable**. All of Contract §3.3 (Stable Time Travel) was satisfied through the inspector and navigation

tools [101] [87], and Contract §6 (Proof of Correctness) was supported by the dev panel which visualizes the hash comparisons and allows step-by-step inspection [102] [103]. A user (or reviewer) could now actually watch determinism happen: e.g., record a session, click verify and see matching hashes, then step through each event and perhaps use the diff to see what each event did. This level of introspection is relatively rare in interactive systems, and it builds confidence that the determinism guarantees aren't just theoretical. Milestone C was validated with tests covering the new functionality (e.g., tests that stepping forward then backward returns to the same state [88], that `getStateAtIndex` returns consistent results for the same index [104], and that diffs correctly identify changes [105]). The features introduced were all about reading and reviewing state, not modifying, which was an intentional choice to keep the model simple (no branching timelines or editing history, which would break the one-true-sequence assumption). This completes the single-user time-travel story for RedByte.

## Milestone D: Operational Validation and System Lock-Down

**Milestone D** was the final step in this series, focusing on validating the entire system in real-world conditions and then "locking" the determinism features as a stable, maintained part of the platform. In earlier milestones, functionality was tested in controlled or simulated environments (lots of unit tests, some integration tests), but Milestone D made sure everything works in an actual browser with actual user interaction, and across different browsers for compatibility.

One aspect was **Browser Environment Validation** – ensuring that features like hashing (which relies on Web Crypto API), large event logs, and the dev panel UI all function in common browsers (Chrome, Firefox, Safari) [106] [107]. For example, since Web Crypto requires asynchronous usage, the team manually verified that performing hashing in the browser yields the expected results and doesn't throw errors. They replaced an earlier Node.js hashing approach with Web Crypto specifically to be compatible with browsers [108], and confirmed that hashing the same input twice yields the same output on each browser [43] [109]. A simple test was to hash a known string "hello world" and compare to a known SHA-256 digest [43], as well as to call the hash function twice and assert equality [110] – this gave confidence that the hashing component is solid in the target environment.

The team also performed an end-to-end test scenario manually in the browser, using the actual UI (Logic Playground with dev panel) [111]. They built a sample circuit (e.g., a 2-to-1 multiplexer circuit with a few nodes and connections), then: 1. Started recording via the dev panel, 2. Interacted with the circuit (toggling some inputs, maybe running a simulation tick or two), 3. Stopped recording, 4. Clicked "Verify Replay" and observed that the panel reported the hashes and a "✓ Deterministic" status [112], 5. Clicked "Export Log" to download the session JSON, 6. Reloaded the page and imported the saved log, 7. Ran verify again on the imported log to ensure it still checks out.

The results were all positive: the dev panel opened without issue, events were recorded (with the first event always being a `circuit_loaded` as expected), the hash comparison showed identical hashes, the exported JSON contained everything (they noted it included the 8 nodes and connections of the test circuit and the one recorded event in that particular trial) [113] [114], and re-importing produced the same hash match [75]. A snippet from their evidence: the live hash and replay hash for that session were identical (e.g., both `f718ecf8...`) and the status displayed "✓ Deterministic" [75]. This confirmed that the whole pipeline works in a user scenario, not just in automated tests.

They also validated **Time Travel Navigation** in the real UI. For instance, after recording and verifying, clicking "Initialize" enabled the step buttons, and if the session had, say, N events, it correctly showed "Event 1 of N" and allowed stepping [115]. For a trivial session (with only 1 event after the initial load), the panel would show "Event 1 of 1" and both step-forward and step-back would be disabled (since there's nowhere to go) [115] – and indeed the implementation handled that gracefully. They noted that a full multi-event time travel test was pending some automated event generation (the simulation engine could eventually generate `simulation_tick` events automatically), but manually they did ensure that stepping didn't throw any errors and correctly updated the state view and counter [116]. The guarantee that you end up in the same initial state after stepping forward and back was inherently validated by these manual tests (and also by automated tests earlier).

Another crucial piece was **Import/Export validation** – ensuring that a saved log file can be reloaded and replayed exactly. They verified that the exported JSON is complete (the initial circuit data was fully preserved, not just a pointer to it) and that metadata like event count was correct [117] [118]. Reloading that JSON into a fresh session and running `verifyReplay` again yielded the same result (hash match) [118]. This demonstrates portability: you could email the JSON file to someone else with RedByte, and they should be able to verify and replay it with identical results. The export format being JSON and self-contained was validated by examining the file – it had the expected structure and content [119] [120].

Having completed these validations, Milestone D proceeded to "lock" the system. **Locking** means declaring that the determinism features are now stable and setting policies to avoid breaking them. The event log format version 1 is now fixed – no changes will be made that alter its interpretation without incrementing the version [121]. Similarly, the determinism contract (v1.0) is now considered **binding**: all the guarantees we've discussed are not experimental; they are promises that the system must continue to uphold [122]. The public APIs that were introduced (functions like `hashCircuitState`, `createRecorder`, `verifyReplay`, `getStateAtIndex`, etc.) are also declared stable [123] [124], meaning future releases of RedByte should not change their function signatures or behaviors in a way that violates determinism or breaks compatibility. For instance, one could add optional parameters or performance improvements internally, but the essence of these APIs should remain the same [125]. This is important for anyone building on RedByte or using it in an educational context – they can trust that scripts or tools using these APIs won't suddenly stop working or produce nondeterministic results after an update.

To ensure these promises hold, the team implemented **regression protection** measures. They have extensive automated test coverage for all the determinism features (unit tests for normalization, hashing, event ordering, integration tests for record→replay→verify, etc.) [126]. Milestone D emphasizes that all contract guarantees are backed by tests [127]. Moreover, they established a practice: any code changes touching determinism must go through a contract compliance checklist (review the contract to see if a guarantee might be affected, run all tests, do manual browser verification with the dev panel) [128]. This kind of discipline is typical in mature systems that have to maintain invariants – similar to how any changes to a cryptographic algorithm would be carefully vetted. Additionally, the documentation (the contract and milestone docs themselves) was completed to be a reference for future contributors [129], so one can't accidentally change something without realizing it might break determinism.

Finally, Milestone D (and the project as a whole) clearly enumerated the **remaining limitations**. Even though everything was validated in mainstream environments, they acknowledge not testing older browsers or mobile browsers [130], and not stress-testing with extremely large circuits or very long event logs [131]. Those are areas outside the current guarantee – performance could degrade or an edge case in

an older browser might appear (though given the nature of the features, serious issues are unlikely, but it's not proven). They also reiterate that security was not addressed: the logs aren't signed, so malicious tampering is a possibility if someone deliberately tries [132]. And the UI remains a developer tool, not a polished end-user feature [133]. These limitations align with what was "not guaranteed" in the contract and are transparently documented.

With Milestone D, RedByte's deterministic subsystem was considered **complete and robust**. The outcome is a system where determinism isn't just a theoretical property but one that's empirically validated and guarded against regressions. At this point, the team signaled readiness to move on to new features built on this deterministic core, as outlined in a post-Milestone-D roadmap [134]. Notably, one of the next steps mentioned is to write a research paper about this very work [135] – effectively, what you are reading now – to situate it in the context of existing literature and share insights with the broader community. Other future directions included bringing these capabilities to end-users (like allowing normal users to save and replay sessions for educational purposes) and exploring minor product features like crash recovery via the recording mechanism [136]. We will touch on some of these in the discussion section, but importantly, all those ideas can now be pursued with confidence because the underlying foundation (determinism contract and implementation) is solidly in place [137].

## Validation

Validation of RedByte's deterministic behavior was multi-faceted, combining **automated testing**, **operational checks**, and **interactive verification**. We outline how each determinism guarantee was verified in practice, to justify the claims of the contract.

**Automated Test Suite:** From Milestone A onward, every component came with dedicated tests. At the unit level, tests ensured that normalization yields consistent outputs (e.g., two differently ordered but equivalent circuit definitions normalize to identical structures) [53]. Hashing functions were tested to confirm that the same circuit produces the same hash every time (and that even across asynchronous calls the result is stable) [138]. The event log format was tested for correctness of serialization – for example, creating an EventLog, converting it to JSON and back, and checking nothing is lost and the version is correct [139]. For the recorder (Milestone B), tests checked that when certain actions are performed in the simulation, the corresponding events appear in the log in order [61]. They also checked that timestamps in events are non-decreasing (monotonic) and that the recorder can be stopped and restarted without merging logs incorrectly [61]. For replay and verification, tests generated small event sequences and asserted that running the replay yields the expected final state and that `verifyReplay` returns `equal=true` when comparing a replay to itself [140]. A particularly important test case is that `verifyReplay` flags a difference when there is any discrepancy – the test suite simulates a divergence (perhaps by altering an event or using a different initial state) and ensures that `verifyReplay().equal` would be false in that case [80]. This gives confidence that our verification method is sensitive to differences and not producing false positives.

For time-travel (Milestone C), tests covered that `getStateAtIndex` for a given index *i* always produces the same snapshot data [104], and that if *j* > *i*, the state at *j* is reachable by starting from state *i* and applying events *i+1…j* (essentially consistency of sequential snapshots). Navigation tests specifically asserted that doing a forward step and then a backward step returns one to the original state [88], which validates the "no history corruption" property. Diffing was tested by constructing scenarios with known changes between states and checking that the diff output lists exactly those changes [105] (e.g., if a node's output flips from 0

to 1 between two snapshots, the diff should contain a SignalChange for that node). They also ran an "integration test" simulating a full time travel session: initialize from a verified log, perform a series of forward steps (say 10), then the same number of backward steps, and confirm the state is back to the starting state and all `canStepForward/Backward` flags behave properly along the way [141]. Passing such a test implies the time navigation logic is coherent.

Beyond unit tests, **integration tests** exercised the whole record→replay cycle. For instance, an automated test might programmatically create a small circuit, simulate a sequence of events (via the same calls the UI would use, but in a test environment), then call `verifyReplay` and expect a deterministic result (equal hashes) [140]. They also tested negative cases – if the log is mutated or events are omitted, `verifyReplay` should report a mismatch [80]. The presence of these tests means any future code change that accidentally, say, iterates a `Map` without sorting or introduces a random factor would likely cause a test to fail (because two replays in the test would yield different hashes). Having tests at multiple levels (unit, integration) that assert the determinism properties creates a safety net ensuring **regression protection** [126].

**Live Operational Validation:** As described under Milestone D, the team manually validated RedByte in actual browsers. This sort of manual QA is crucial for aspects that are hard to fully simulate in automated tests, such as the proper functioning of the Web Crypto API across browsers or the behavior of the UI panel. The report from these tests was positive: e.g., in Chrome and Firefox, recording and replaying a session resulted in identical hashes and a "Deterministic" status in the UI [113] [75]. They even took a screenshot (as evidence) of the dev panel showing the same hash for live and replay runs [142]. While a screenshot is not formal proof, the identical hash string strongly indicates correctness, and doing this on multiple browsers suggests cross-browser consistency. They did not report any differences between Chrome's and Firefox's handling of our features, which is a good sign that our reliance on standardized APIs (like Web Crypto) paid off.

**Hash Comparison as Ground Truth:** One might ask, how do we *know* the states are identical, beyond just our expectation? The answer is the hash comparison. In every test and during every verification in the UI, the SHA-256 hash of the entire circuit state is the arbiter. If anything, even a single bit in the state, were different between two runs, the 256-bit hash would (with overwhelmingly high probability) change. By designing our verification around a cryptographic hash, we gain very high confidence in identity of states. A non-cryptographic approach (like comparing object graphs directly) could be error-prone or might ignore some subtle difference. The hash is a single, easy-to-check indicator. In practice, we never observed hash mismatches in verified sessions unless we *intentionally* injected a difference (like as part of a test). That suggests RedByte's determinism is airtight for the scenarios tested.

**User-Level Verification:** Another form of validation is that *users themselves can verify the determinism*, thanks to the dev panel. The contract even provides a mini "how to verify" guide: open the app, press the hotkey, record an interaction, click verify, and see that `liveHash === replayHash` [143]. We tried this procedure as well and found that it consistently yields a checkmark for determinism (assuming no bugs in the user's circuit logic itself). This is quite a unique aspect: typically, end-users or developers have to trust the system's claim of determinism, but here they can witness it. It's akin to a scientific experiment that anyone can reproduce – a strong validation technique in its own right.

**Limitations of Validation:** We acknowledge that while our testing was thorough within the intended scope, it wasn't exhaustive over all possible environments. As noted, older or obscure browsers haven't been tested; it's possible (though unlikely) that some JavaScript quirk or Web Crypto bug in, say, a very old Safari

could cause an issue. We also did not simulate extremely large circuits or logs in the browser – performance might degrade or even cause memory issues for huge logs (thousands of events). However, these scenarios are outside normal use (a user would rarely perform thousands of manual interactions in one go). In terms of logical validation, we have not performed a formal proof of determinism (e.g., using a theorem prover); our approach has been empirical. We rely on the strength of tests plus the determinism of the JavaScript single-threaded model. It's worth noting that JavaScript itself is deterministic for a given program (no random scheduling, etc., if you don't use Web Workers or certain non-deterministic APIs). Since our design confines all state changes to be driven by our event loop and data structures, we inherit some determinism from the environment too.

In conclusion, the validation efforts give us high confidence that RedByte meets its determinism contract: every guarantee listed has a corresponding verification. All contract guarantees are backed by *executable tests or demos* [127] . This comprehensive validation is what allows us to assert that determinism in RedByte is "not merely asserted — it is proven" [144] (in a practical sense of proof). The system's behavior has been checked from multiple angles, and as of the completion of Milestone D, we consider the determinism contract verified in the implemented system.

## Comparison to Prior Work

RedByte's approach to deterministic interactive computation touches on elements of debugging, state replay, and educational tooling. Here we position it relative to notable prior work in these areas, highlighting differences in goals and novelty.

**Record/Replay Debuggers (rr, TTD, etc.):** Traditional record-and-replay debuggers operate at the level of program execution, recording low-level nondeterministic events to allow exact replays. Mozilla's **rr** is a prominent example: it logs all inputs to a process and CPU nondeterminism, producing a trace that can be replayed under gdb with reverse execution [4] . rr's design shows that deterministic replay is possible even for complex C/C++ programs, but the cost is the complexity of capturing everything – system calls, thread schedules, signals, etc. Moreover, rr is used offline: one must run the program under rr, then later debug using the trace. RedByte in contrast operates at the application semantics level. We do not record every memory access or thread context switch (in fact, our system is single-threaded in JavaScript); we only record *domain-specific events*. This makes our logs much smaller and more interpretable. Another difference is **accessibility**: rr is a developer tool requiring a special run, whereas RedByte's replay is built into the normal operation of the app (especially with the dev panel UI in Milestone C, it's one keypress away). RedByte could be used not just by developers but potentially by end users (for example, a teacher saving a circuit demo) – something rr is not designed for. Microsoft's **Time-Travel Debugging (TTD)** for WinDbg similarly records execution traces to allow backward stepping in debugging sessions [5] . It integrates with the debugger to let you query state history. Our time-travel in RedByte is conceptually similar in that one can go forward/backward, but TTD is focused on imperative code debugging (stepping through lines of code and machine states). RedByte's time-travel is higher-level: stepping through *user events* in a simulation. The major novelty of RedByte relative to these is treating replay as a first-class *feature* of the application, not just a debugging technique. We enforce determinism from the start, whereas rr/TTD assume the program may not be deterministic and thus they intervene to capture enough data (e.g., rr uses CPU performance counters and other tricks to deterministic-ize a run). RedByte doesn't need to record CPU-level details because the program logic itself has been constrained to be deterministic.

**Time-Travel UI Debuggers (Elm, Redux DevTools):** The Elm language's time-travel debugger and **Elm Reactor** are one of the closest analogues to RedByte's philosophy. Elm can record all user inputs to a pure-functional program, and because of Elm's architecture, replaying those inputs deterministically reconstructs the program state [9] [145]. Elm even supports features like snapshotting state to avoid reprocessing too many events and the ability to modify code and replay the same inputs to see how behavior changes [146] [147]. RedByte's determinism contract is in the same spirit—Elm guarantees that pure functions with the same inputs yield same outputs (trivial by definition of purity), and RedByte guarantees the whole simulation with same events yields same final state. However, Elm's solution is language-specific and aimed at *debugging during development*. You use Elm's time-travel in debug mode, but you wouldn't ship it as part of a production user experience. RedByte's determinism, while showcased in a dev panel currently, is designed such that it could be a user-facing capability (e.g., "Save your session and replay it later" in a learning context). Additionally, Elm leverages purity—something ensured by its compiler and runtime. RedByte had to achieve a similar effect in a less inherently pure environment (JavaScript with mutable state), which required explicit measures like state normalization and event logging. Redux DevTools for React apps is another similar tool: it logs every Redux action and allows stepping through the sequence of actions to inspect past states [7]. In fact, Redux DevTools and RedByte both generate a history of state changes that can be exported and shared to reproduce bugs [148]. One key difference is that Redux DevTools relies on developers following the Redux pattern (pure reducers, no external side-effects on state). RedByte's novelty is that it *enforces* a pattern ensuring determinism (through its contract and design) rather than relying on convention. Also, RedByte's verification via hashing has no direct equivalent in Redux – in Redux, you assume determinism; in RedByte, you can prove it for each session.

**Educational Simulators and Notebooks:** RedByte can be viewed as a domain-specific application (a logic circuit simulator) with advanced reproducibility features. Traditional digital logic simulators (like Logisim, Multisim, etc.) don't emphasize replay; they are typically used live, and while you can often toggle inputs and see outputs, there's no standard mechanism to record that toggling sequence and play it back. Some simulators might let you write a script or testbench to automate inputs, but doing so manually is outside the typical user flow. RedByte, by building in event recording, essentially turns any interactive usage into a script that can be rerun. This is similar in ethos to *literate programming notebooks* or scientific workflows where one strives for reproducibility: e.g., Jupyter notebooks allow rerunning the same code cells to reproduce analysis, but interactive widgets in notebooks are often not reproducible without manual steps. RedByte could be seen as bringing the reproducibility ideals of scientific computing into an interactive GUI context. In education, this is novel: a student in a digital logic class could perform an experiment (simulate a circuit with certain input changes over time) and then submit not just the final circuit design but the *recorded simulation* to the instructor. The instructor can replay it exactly and see the circuit's behavior unfold. This provides a richer communication of process, not just result, which is rarely captured in today's tools.

**Interactive Systems Research:** In the research community, there have been many explorations of making programs more transparent and debuggable. For example, the concept of an **omniscient debugger** (also called reverse debuggers or time-travel debuggers in academia) has been discussed since the 1970s and 80s, including experiments with reversing execution in languages like Prolog (which naturally has backtracking). More recently, HCI and PL research has looked at live programming environments where you can scrub through program execution or timelines (such as Bret Victor's work on Inventing on Principle, which demonstrated instant feedback and the ability to "play" with time in simple simulations). RedByte contributes to this space by showing a **concrete, implemented example** where an interactive system (with a complex state like a circuit simulator) is made fully deterministic and introspectable without special

hardware or custom OS support – just through careful software architecture. It demonstrates that one can retrofit strong reproducibility into an interactive domain that isn't purely functional. RedByte's approach could inspire similar determinism contracts in other domains, say a graphical editing tool that records editing commands or a simulation environment for physics experiments. The idea of treating the UI's input event stream as the *log of truth* and basing replay on it is applicable broadly. Unlike some prior research prototypes that remain conceptual, RedByte is deployed in a real web app, which underscores its practicality.

In summary, RedByte stands at an intersection of ideas: like rr/TTD it offers deterministic replay, but at a high level and integrated into the app; like Elm/Redux it offers time-travel debugging, but with stronger guarantees and generality beyond pure functions; like educational tools it provides a sandbox for learning, but with a novel "flight recorder" running in the background. Its novelty lies in **synthesizing these elements** into a cohesive system and treating determinism as a core feature rather than a hack or an afterthought. We believe this is a step forward in how interactive applications can be built – acknowledging that the sequence of user interactions is as important a product of the application as the final output, and ensuring that sequence can be re-examined, shared, and trusted.

## Discussion and Limitations

While RedByte demonstrates the feasibility and value of deterministic interactive computation, it is important to discuss the **limits of this approach** and areas that we deliberately chose not to address (at least in this initial design).

**Not a General Debugger:** RedByte's determinism features might look like debugging tools, but they are specific to the context of the logic simulator. We did not build a general-purpose omniscient debugger for arbitrary code – our replay and time-travel work at the level of circuit state and pre-defined event types, not at the level of source lines or machine instructions. This was a conscious choice: by limiting scope to domain events, we kept the system simpler and more predictable. Traditional debuggers allow breakpoints, arbitrary inspection, modification of state mid-execution, etc. RedByte's time-travel does not allow modifying history or injecting new events out-of-order. You can't, for example, *branch* the timeline and try a different input after event 5 – you would have to create a new session and record that as a separate log. This is in line with our philosophy of determinism as a contract: the recorded log is an authoritative record of one timeline, and we don't deviate from it within that context. If users need a different scenario, they record a different log.

**No Multi-User or Network Guarantees:** RedByte is a single-user system running in one browser instance. We explicitly excluded any notion of synchronizing determinism across multiple users or over a network [24]. In a collaborative setting (imagine two people jointly editing a circuit in real-time), ensuring determinism would be far more complicated – one would have to totally order all events from all users, handle network latency, and possibly deal with conflicts. Some distributed systems achieve *eventual consistency* or *deterministic merges*, but that is outside RedByte's scope. If RedByte were extended for collaboration, likely the event log would become a shared log and one would need a protocol to agree on event ordering (which is a whole research area in itself, e.g., CRDTs or operational transforms). For now, RedByte avoids these complexities by focusing on the scenario of one user creating a deterministic log that others can later replay, but not concurrent interaction. This is a limitation, but an intentional one because it aligns with our primary use cases (education, debugging, sharing sequences after the fact, not simultaneous editing).

**Performance and Scale:** We have not optimized RedByte for very large circuits or extremely long sessions. The determinism features introduce some overhead – for example, sorting objects for normalization is O(n log n) in the size of the circuit [149], hashing is cryptographically secure but not constant-time (it processes the entire state), and replay means essentially re-running the simulation potentially many times (especially if doing time-travel back and forth). In our testing with reasonably sized circuits (tens of nodes, maybe hundreds of events), performance was more than adequate (sub-second for most operations). However, if someone tried a circuit with thousands of elements or recorded hundreds of thousands of events (say they left the recorder on for hours), the experience might slow down. We did not implement optimizations like Elm's snapshotting mechanism [150] [151], which could be a future improvement to handle large histories. The contract explicitly does not guarantee real-time performance [19], and indeed one might experience slow stepping if the log is huge [152]. The focus was correctness over speed; thus, in practical use, one might need to keep an eye on log sizes. For typical interactive use cases (which tend to be short-ish sessions), this is fine, but it's a limitation to note.

**Tamper Detection and Security:** RedByte's replay will happily accept any event log JSON and execute it. We do minimal validation on the input (basically checking the version field and that the JSON structure is correct). There is no built-in signature or checksum to verify that a log wasn't modified by a third party. The contract notes that we do not attempt to prove authenticity, only identity [21]. So if someone shares a log, the recipient must trust its source. For example, a malicious log could conceivably be constructed to exploit a bug in the logic engine or just to mislead (though since logs just toggle circuit inputs, the harm is limited to the circuit simulation itself, not the broader system). If RedByte logs were used in an assessment context (say for homework submission), one could cheat by editing the log to produce a desired outcome, and unless there's an external check (like the instructor also has the expected hash), this could go unnoticed. Addressing this would require some sort of cryptographic signing of logs, or embedding a secure hash of the log's content in a trusted place. We left this out of scope because our aim was reproducibility, not security. In a future iteration, if RedByte logs become a medium of exchange in untrusted scenarios, adding signatures would be advisable. But that again raises the complexity (key management, etc.), so we kept things simple for now.

**UI and Usability:** The determinism dev panel is functional but not user-friendly in a polished sense [153]. It's clearly targeted at developers or advanced users who understand the concept of event logs and hashes. We have not done user studies on how non-technical users might use or benefit from determinism. One can imagine a more friendly UI for general users: e.g., a "Replay Demo" button on a circuit that just plays back the events with a nice animation, without exposing hashes or requiring a Ctrl+Shift+D. Our roadmap includes possibly promoting the record/replay to a user feature (Phase E2: Education UX) [136]. At that point, we'd need to wrap these capabilities in a more intuitive interface and guide users on how to use them (for instance, a tutorial overlay explaining "You can save your session and replay it later" etc.). The current limitation is that the feature is somewhat hidden (deliberately, as it's dev-focused) and not optimized for discoverability or novice use.

**Assumed Deterministic Engine Implementation:** One subtle point: RedByte's determinism assumes that the underlying logic simulation engine (the code that updates circuit state on each tick or input change) is itself deterministic given the same inputs. We designed it to be so, and as part of the contract we restrict to the *reference* engine only [22]. If someone wrote a custom engine for the same circuits (say in a different programming language or using WebAssembly for speed), there's no guarantee it would produce bit-for-bit identical state trajectories, especially if floating point or other complexities come in. So RedByte determinism is only as strong as the engine's consistency. We mitigate this by keeping the engine simple

(only boolean logic, largely combinatorial updates, etc.). If in the future RedByte extends to simulate more complex analog behavior or random events, determinism would need reevaluation. In effect, we've proven determinism for the current scope (logic circuits with binary states). The limitation is that we cannot claim determinism for any arbitrary simulation plugged into RedByte's framework; one would have to ensure any new simulation logic also adheres to deterministic practices.

**Not a Network/Distributed Log System:** Our export/import mechanism allows sharing logs, but we do not provide a central repository or version control for logs. Each log is a standalone artifact. There's no built-in feature to, say, merge two logs or compare two different users' logs. If multiple logs are collected, it's up to the user to organize and identify them. This is simply a non-goal for now, but worth noting if one compares RedByte to, say, a collaborative science platform or version-controlled notebook. RedByte's logs are more akin to short recordings or demos, not a full provenance tracking system (although conceptually, a series of logs could be chained if one wanted to represent a branching scenario – but we have not explored that).

In reflecting on these limitations, we see them largely as *scope definitions* rather than flaws. By narrowing focus, we achieved a strong result in that area. Many of the above could be avenues for future improvement: - For multi-user determinism, one could attempt a sync protocol or at least use determinism to ease merging logs from different users (if their interactions were separate). - For performance, techniques like state snapshotting (as Elm does) or incremental hashing (maintaining hash as state changes rather than recompute from scratch) could be added to handle larger logs more smoothly. - For security, adding a digital signature to the exported log (and verifying it on import) could ensure authenticity. This would be straightforward using standard cryptography, but managing keys/certs in a browser context might require user intervention or tying logs to user accounts. - For UI, integrating these features into the main interface with clear language ("Save Session", "Replay Session") could open them up to a broader audience, which in turn would generate feedback on how useful determinism is for non-developers.

**Novelty and Impact:** It's worth highlighting why we believe treating determinism as a semantic contract is impactful. In many interactive applications, determinism is an afterthought – something you might try to retro-fit if you need reproducibility for debugging. RedByte shows that by designing for determinism from the ground up, you gain not only easier debugging but also *new capabilities* (like shareable interactive recordings) essentially for free. It turns out that the discipline needed for determinism (no hidden state, clear separation of concerns, logging inputs) is also just good software design in general. It forces one to make the system more observable and testable. In that sense, even if one didn't care about replay, following a determinism contract could improve reliability. We've encapsulated these design rules in our contract and implementation – which others could use as a template. The hope is that RedByte can influence other projects to consider a determinism contract approach, especially in educational software or any scenario where reproducibility would enhance collaboration.

To sum up, RedByte currently has clear boundaries on what it does: single-user, deterministic logic simulation with verifiable replay and time-travel, running in a modern browser. It achieves this well, as evidenced by our validation. The limitations outlined are mostly conscious decisions to keep the problem tractable. We view the work as a strong foundation upon which further research and development can build – whether that's extending determinism to new domains or integrating these ideas into user-facing features. The next section concludes with final thoughts and points to future directions.

## Conclusion

This paper presented **RedByte**, a browser-based interactive system that achieves *deterministic, replayable computation* through a rigorous semantic contract and practical implementation. By treating determinism as a first-class design goal, RedByte demonstrates that even highly interactive, UI-driven applications can have the same reproducibility traditionally reserved for pure functions or batch programs. Key to our approach is logging user events as the sole drivers of state change and enforcing a canonical representation of state, so that "same inputs, same outputs" becomes an invariant of the system [1]. We showed how cryptographic hashing can serve as an efficient test for execution identity [2], enabling automated and user-visible verification of determinism. We also introduced interactive time-travel debugging features that leverage the deterministic model to allow seamless navigation through past states [14], something usually extremely hard to get outside of specialized debugging tools.

Our evaluation of RedByte, via extensive tests and live trials, confirms that the system meets its determinism guarantees: any recorded session can be replayed bit-for-bit the same (with matching SHA-256 hashes as evidence) [112], and intermediate states are consistent upon multiple replays. The success of RedByte's approach suggests that **deterministic interactive computation is not only possible but practical in modern web environments**, without requiring exotic tools or sacrificing usability. We believe this work bridges a gap between programming languages theory (which often assumes deterministic pure functions) and real-world interactive software, showing a path to make the latter more predictable and transparent.

In terms of novelty, RedByte's contribution is to **elevate determinism to a user-level feature** rather than a behind-the-scenes mechanism. This stands in contrast to prior record/replay systems which were mostly for internal debugging. In RedByte, a developer (and in the future, perhaps any user) can literally click a "Verify" button and get a guarantee of deterministic behavior for their session. This kind of built-in assurance could be valuable in many domains: imagine collaborative design tools where any sequence of edits can be shared and exactly reproduced, or educational platforms where an experiment's procedure can be disseminated as an interactive log. RedByte offers a template for how to build such systems.

Looking ahead, there are several exciting directions. One is applying RedByte's determinism framework to **other types of simulations or interactive content** – for example, a physics sandbox or an educational game – to enable verified replays in those contexts. Another direction is improving the **user experience** around determinism: making it so intuitive and useful that even non-programmers start to expect that their software can "time-travel" or provide replayable logs. On the research side, formal verification of parts of RedByte's determinism (proving properties of the event log or the engine) could provide an even deeper guarantee. And while we intentionally avoided distribution, exploring a controlled extension of this work to collaborative environments (perhaps synchronizing event logs among multiple clients in a deterministic way) could combine reproducibility with multi-user interactivity.

In conclusion, RedByte shows that by designing with a determinism contract in mind, we can build interactive systems that are **predictable, debuggable, and shareable** in ways not typically seen. We encourage other system designers to consider adopting similar contracts for their domains, as the benefits for reliability and user understanding are significant. Determinism need not be left to the realm of theoretical computer science – as RedByte illustrates, it can be engineered into everyday software, bringing us a step closer to truly intelligible and reliable computing experiences.

---

1 2 3 8 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 27 28 29 30 31 32 33 41 99 100 127 143 144 CONTRACT.md
https://github.com/swaggyp52/redbyte-ui-genesis/blob/9faa1d24030f0ee1ce0936044e28b4980c2b5635/docs/determinism/CONTRACT.md

4 rr (debugging) - Wikipedia
https://en.wikipedia.org/wiki/Rr_(debugging)

5 Time Travel Debugging Overview - Windows drivers | Microsoft Learn
https://learn.microsoft.com/en-us/windows-hardware/drivers/debuggercmds/time-travel-debugging-overview

6 9 145 146 147 150 151 Time Travel made Easy
https://elm-lang.org/news/time-travel-made-easy

7 148 Time Travel in React Redux apps using the Redux DevTools | by Onsen UI & Monaca Team | The Web Tub | Medium
https://medium.com/the-web-tub/time-travel-in-react-redux-apps-using-the-redux-devtools-5e94eba5e7c0

26 34 35 36 37 38 39 40 42 45 46 47 48 49 50 51 52 53 54 138 139 149 milestone-a.md
https://github.com/swaggyp52/redbyte-ui-genesis/blob/9faa1d24030f0ee1ce0936044e28b4980c2b5635/docs/determinism/milestone-a.md

43 44 74 75 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 128 129 130 131 132 133 134 135 136 137 142 milestone-d.md
https://github.com/swaggyp52/redbyte-ui-genesis/blob/9faa1d24030f0ee1ce0936044e28b4980c2b5635/docs/determinism/milestone-d.md

55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 76 77 78 79 80 140 milestone-b.md
https://github.com/swaggyp52/redbyte-ui-genesis/blob/9faa1d24030f0ee1ce0936044e28b4980c2b5635/docs/determinism/milestone-b.md

81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 101 102 103 104 105 141 152 153 milestone-c.md
https://github.com/swaggyp52/redbyte-ui-genesis/blob/9faa1d24030f0ee1ce0936044e28b4980c2b5635/docs/determinism/milestone-c.md